Contents

Mock Interview — Senior DevOps Engineer **1**
Instructions for Self-Practice . . . 1
Round 1: Architecture & Design (45 min) . . . 1
Q1: System Design . . . 1
Q2: Incident Scenario . . . 2
Q3: Trade-offs . . . 2
Round 2: Technical Depth (30 min) . . . 2
Q4: Kubernetes Networking . . . 2
Q5: Prometheus Deep Dive . . . 3
Q6: Security . . . 3
Round 3: Behavioral (20 min) . . . 3
Q7: Conflict Resolution . . . 3
Q8: Failure . . . 4
Scoring Rubric . . . 4
30-Day Interview Prep Plan . . . 4

Mock Interview — Senior DevOps Engineer

Instructions for Self-Practice

1. Cover the answers section below each question
2. Set a 3-minute timer and answer out loud (record yourself)
3. Uncover the answer and compare
4. Rate yourself: Depth (1-5) · Clarity (1-5) · Completeness (1-5)
5. Repeat weak areas until consistent 4+ scores

Round 1: Architecture & Design (45 min)

Q1: System Design

> **"Design the CI/CD infrastructure for a company deploying 50 microservices 30 times a day to Kubernetes. They have dev/staging/production environments."**

Ideal answer covers:

Clarifying questions to ask first: - Team size? (affects pipeline parallelism needs) - Monorepo or polyrepo? - Kubernetes managed (EKS/GKE) or self-managed? - Any compliance requirements (SOC 2, PCI)? - Current tooling or greenfield?

Architecture decision: - GitOps model: Git is source of truth, ArgoCD pulls desired state - GitHub Actions for CI (build, test, scan, push image) - ArgoCD for CD (sync manifests from Git to cluster) - Separate pipelines per service but shared runner pool

Pipeline flow: 1. Developer pushes → PR created 2. CI: lint → unit test → SAST → build image → Trivy scan → push to ECR with SHA tag 3. CI updates Helm values file in GitOps repo with new image tag 4. ArgoCD auto-syncs staging within minutes 5. Smoke tests run against staging 6. Production promotion: PR to GitOps repo → reviewer approves → ArgoCD syncs production

Environment isolation: - Separate K8s namespaces with RBAC - Separate AWS accounts for production (blast radius containment) - Separate ArgoCD projects with access control

Observability: - Deployment annotations in Grafana → correlate deploys with metric changes - ArgoCD sync status → alert if sync fails or drift detected - DORA metrics tracked: deployment frequency, lead time, MTTR, change failure rate

---

Q2: Incident Scenario

> **"At 2am you get paged: 'checkout is down, all purchases failing.' You are the IC. Walk me through your response."**

Ideal answer covers:

First 2 minutes: - Open incident channel `#inc-20260525-checkout-down` - Assign Tech Lead if available, set Comms role - Set severity: SEV-1 (revenue-impacting, all users) - Acknowledge alert to stop repeat pages

Investigate (3-10 min): - Check Grafana dashboard: error rate, latency, traffic drop - `kubectl get pods -n production` — any CrashLoopBackOff, OOMKilled? - `kubectl get events -n production --sort-by=.lastTimestamp` - Check recent deployments: ArgoCD history, deployment annotations on dashboard - Check downstream dependencies: payment API, database response times

Mitigation before root cause (10-20 min): - If recent deploy: `kubectl rollout undo deploy/checkout -n production` — rollback first, investigate later - If database: check connection pool, query times, read replica lag - Scale up if traffic spike: `kubectl scale deploy/checkout --replicas=10` - Update status page and stakeholder channel every 10-15 minutes

Recovery confirmation: - Error rate returns to baseline (< 0.1%) - Successful end-to-end checkout test - Monitor for 10 minutes before declaring resolved

Post-incident: - Blameless post-mortem within 48 hours - Action items with owners and due dates - Update runbook with new failure mode

---

Q3: Trade-offs

> **"When would you choose Blue-Green over Canary deployment? What about Feature Flags?"**

Blue-Green — choose when: - Need instant rollback capability (< 30 seconds, just flip the load balancer) - Stateless services only (sessions don't cross environments) - Full environment parity required for testing - DB migrations already applied and backwards-compatible - Cost: requires double infrastructure during switch

Canary — choose when: - Need to validate behavior with real production traffic gradually - Can monitor error rate and auto-rollback on threshold breach - Traffic shaping at ingress level (NGINX weighted routing or Argo Rollouts) - More gradual risk: 5% → 25% → 100% - Takes longer but catches issues before full blast radius

Feature Flags — choose when: - Want to decouple deploy from release (deploy the code, turn on the feature separately) - A/B testing different implementations - Emergency kill switch needed (turn off feature without redeploying) - Targeting specific users/cohorts/regions - Dark launching: code in production but feature off for all users

Real answer: Use all three together. Deploy with canary, controlled by feature flags, with blue-green for rollback. The combination gives maximum safety with fine-grained control.

Round 2: Technical Depth (30 min)

Q4: Kubernetes Networking

> **"Explain exactly what happens when a pod in namespace A calls `http://myservice.namespace-b.svc.cluster.local:80`."**

Answer: 1. Pod's DNS resolver (stub resolver) queries `169.254.20.10:53` (or CoreDNS ClusterIP on port 53) 2. CoreDNS looks up `myservice.namespace-b.svc.cluster.local` 3. CoreDNS returns the Service's ClusterIP (e.g. `10.96.45.12`) 4. Pod initiates TCP connection to `10.96.45.12:80` 5. The packet hits `iptables`/`ipvs` rules installed by `kube-proxy` on the source node 6. `kube-proxy` DNAT rule rewrites destination: `10.96.45.12:80` → one of the pod IPs in the Endpoints list (e.g. `10.244.2.5:8080`) 7. Packet is routed to the destination node via the CNI overlay network (VXLAN/BGP depending on CNI) 8. Pod receives the request on its actual IP

What can go wrong: - DNS failure: CoreDNS pod down, or `ndots:5` search domain causing excessive DNS queries - Empty Endpoints: pod label selector doesn't match Service selector - NetworkPolicy blocking: source

pod or destination pod has restrictive policy - kube-proxy iptables rules stale: `kube-proxy` pod crashed or node network issue

---

Q5: Prometheus Deep Dive

> **"We have 100 microservices. How do you prevent Prometheus from running out of memory? What is cardinality and how do you control it?"**

Answer: Cardinality = total number of unique time series = unique combinations of all label values.

What causes high cardinality: - User IDs, order IDs, session IDs as labels → millions of unique series - URLs with path parameters: `/users/12345/orders` as a label - Pod names in labels (rotate constantly) - Timestamps or request IDs

Why it's dangerous: Each unique series requires memory in the TSDB. A Prometheus with 10 million series might need 10-20 GB RAM just for the head block. Series churn (new series created constantly) makes it worse.

Controls: 1. **Relabeling rules** in Prometheus scrape config: drop high-cardinality labels before ingestion 2. **Recording rules**: pre-aggregate expensive PromQL expressions, store as lower-cardinality series 3. **Metric labels review**: enforce label naming standards in code review 4. **Cardinality monitoring**: alert when a job's series count exceeds threshold 5. **Remote write filtering**: use `write_relabel_configs` to drop before sending to Thanos 6. **Tool**: `mimirtool analyze` or Grafana's Cardinality Management page shows worst offenders

For 100 services: - Federated Prometheus: one Prometheus per namespace/team, global Prometheus scrapes aggregated metrics only - Or: Thanos/Cortex/Mimir for horizontally scalable storage — multiple Prometheus instances, Thanos Querier provides global view

---

Q6: Security

> **"A developer accidentally committed an AWS secret key to the public GitHub repo 2 hours ago. What do you do?"**

Immediate (minutes): 1. Rotate the key immediately — AWS console → IAM → Security credentials → make inactive + delete 2. Create a new access key if the service still needs it (or switch to IRSA/role-based) 3. Check CloudTrail for API calls with the compromised key in the last 2 hours 4. Look for: unusual regions, new IAM users created, new EC2 instances, S3 data access/exfiltration

Clean up Git history: 5. `git filter-repo` (preferred) or BFG Repo-Cleaner to remove secret from all history 6. Force-push to overwrite remote history (coordinate with team — they need to re-clone) 7. GitHub has a secret scanning feature that may have already flagged it — check security alerts

Communicate: 8. Notify security team per incident process 9. Document: what was exposed, what was accessed, what was done 10. If customer data was accessible: legal/compliance review for breach notification

Prevent recurrence: 11. Enable GitHub Secret Scanning and push protection (blocks commits with secrets) 12. Add gitleaks as pre-commit hook for all developers 13. Eliminate static credentials entirely: switch to IRSA for EC2/EKS, OIDC for CI/CD

Round 3: Behavioral (20 min)

Q7: Conflict Resolution

> **"Tell me about a time you disagreed with your manager about a technical decision."**

Framework: - State the disagreement clearly and factually (don't be vague) - Show you heard their position and tried to understand their reasoning - Explain how you raised your concern (data, not opinion) - Show you were willing to accept the decision even if not your preference - Explain the outcome honestly

Red flags to avoid: - “I was right and they eventually realized it” (arrogant) - “I just did what they said” (no backbone) - “We never disagree” (not credible)

Strong structure: > “My manager wanted to skip the canary phase and do a direct blue-green release for a critical payment service change. I was concerned because the new code path had significantly different DB query patterns we hadn’t load-tested. I prepared a short risk analysis — estimated 15 minutes of canary observation would tell us if queries were performing correctly. I proposed we time-box it: 15-minute canary at 5%, if no latency spike we go straight to 100%. They agreed. We did find a slow query during the canary that would have caused degradation at full load. Caught it with 5 minutes of impact vs potentially 30 minutes.”

Q8: Failure

“Describe the biggest technical mistake you’ve made in your career.”

What interviewers are evaluating: - Self-awareness (can you identify your own failures?) - Maturity (do you blame others, or own it?) - Learning mindset (what changed as a result?) - Judgment (is the mistake proportionate to seniority — was it actually a hard problem?)

Structure: 1. Describe what happened clearly and specifically 2. Own your part without excessive self-flagellation 3. Describe the impact honestly (affected X users for Y minutes) 4. Describe what you changed: process, tooling, behavior 5. End with the systemic fix, not the personal lesson

---

Scoring Rubric

Score	Technical	Communication	Problem Solving
5	Deep expertise, tradeoffs articulated clearly	Clear structure, concise, no rambling	Systematic, asks good questions
4	Solid knowledge, one or two gaps	Good clarity, minor organization issues	Logical approach, finds answer
3	Surface knowledge, can’t go deep	Understandable but disorganized	Gets there with hints
2	Misconceptions, significant gaps	Hard to follow	Struggles without guidance
1	Incorrect or no knowledge	Cannot explain	No structured approach

Target for Senior DevOps role: 4+ on all dimensions

30-Day Interview Prep Plan

Week	Focus	Practice
1	K8s deep dive + Linux internals	2 mock sessions on architecture
2	Cloud (AWS) + Terraform	Design a multi-region architecture
3	Monitoring + SRE + Security	Incident simulation exercise
4	Behavioral + full mocks	3 full end-to-end mock interviews

Daily habits: - 30 min: Read one topic from notes (rotate domains) - 15 min: Practice one PromQL query / kubectl command set - 15 min: One behavioral question, answered out loud